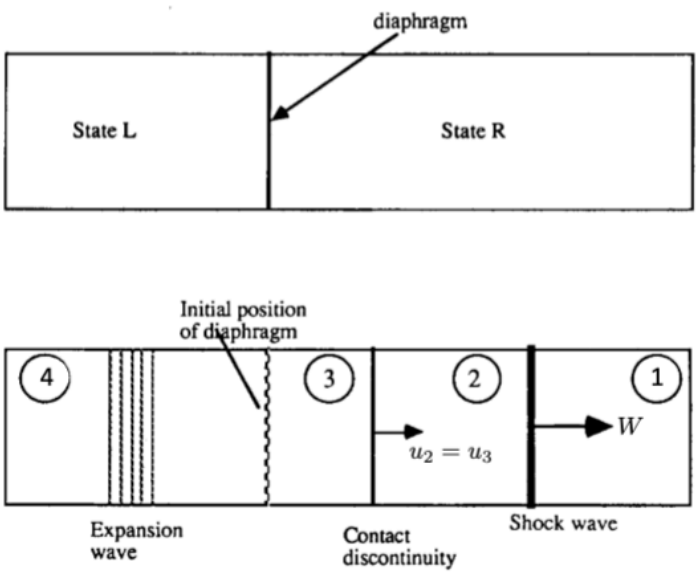
Anne Jumper
ME193: Computational Fluid Dynamics
April 17, 2026

Numerical Analysis of One-Dimensional Unsteady Compressible Flow: The Shock Tube Problem

I. Introduction to the Shock Tube Problem



Region 1: Undisturbed portion of driven section, with original values of u_1=0, P_1, ρ_1, and T_1

Region 2: Processed by shock waves propagating through it, $P_2 > P_1$, $u_2 > 0$, $\rho_2 > \rho_1$.

Region 3: Processed by expansion waves (isentropic waves), $P_3 = P_2$, $u_3 = u_2$, $s_3 = s_2$, so $T_3 = T_2$ and $\rho_3=\rho_2$.

Region 4: Undisturbed portion of driver section, with original P_4, ρ_4, T_4, $u_4 = 0$.

Fig 1. Diagram illustrating how a shock wave propagates and affects physical properties of the gas in different regions.

The shock tube problem or Riemann problem, is an important one-dimensional problem in computational fluid dynamics as it validates the performance of numerical schemes where there are sharp discontinuities and instantaneous physical changes present. The physical setup consists of a long, closed cylindrical tube divided into two sections by a thin diaphragm. One side, called the driver section, is filled with a high-pressure gas, while the other side, the driven section, contains gas at a lower pressure. At time $t = 0$, the diaphragm is instantaneously ruptured, resulting in a rapid equalization process. This process produces three primary wave structures: a shock wave that travels into the low-pressure region from the high-pressure region (to the right), an expansion wave that moves into the high-pressure region from the low pressure region (to the left), and a contact discontinuity between the shock and the expansion wave. At the contact discontinuity, pressure and velocity remain constant across the boundary, but density and temperature undergo a sharp jump. In an ideal inviscid scenario, these transitions are infinitely sharp. However, in real scenarios, fluid viscosity and thermal conductivity smooth out sharp changes in physical properties. This report models this system using the MacCormack scheme, compares second and fourth-order spatial discretization and various mesh refinements, and analyzes the transition from the idealized inviscid case to a more realistic viscous model.

II. Governing Equations and Physical Modeling

The dynamics of the gas within the tube are governed by the one-dimensional, unsteady, compressible Navier-Stokes equations. To ensure the numerical scheme correctly calculates the position and speed of the shock wave, the equations track the flux of mass, momentum, and energy through the whole system. This ensures these quantities remain balanced across the shock so the model captures the sharp discontinuities. The system is described by the relation where the time rate of change of the state vector plus the spatial gradient of the flux vector equals the source vector. Mathematically, this is:

$$\frac{\partial U}{\partial t} + \frac{\partial F}{\partial x} = J$$

The state vector, *U,* contains the variables that are conserved (density, momentum, and total energy per unit volume). The flux vector, *F,* represents the transport of these quantities through the tube, and the source vector, *J,* incorporates the viscous and heat transfer effects (as sources or sinks of momentum and energy). These vectors are defined as:

$$U = \begin{bmatrix} \rho \\ \rho u \\ \rho E \end{bmatrix}, \quad F = \begin{bmatrix} \rho u \\ \rho u^2 + P \\ (\rho E + P)u \end{bmatrix}, \quad J = \begin{bmatrix} 0 \\ \frac{\partial \tau}{\partial x} \\ \frac{\partial (u\tau - q_x)}{\partial x} \end{bmatrix}$$

In these expressions, ρ is the fluid density, u is the velocity, P is the static pressure, and E is the internal energy. The fluid is assumed to be an ideal gas, which allows for the implementation of the Ideal Gas Law $P = \rho RT$, where R is the gas constant for air, $286.9\ J/kg{\cdot}K$. The internal energy is related to temperature through the constant volume specific heat C_v as $e = C_vT$. To model the internal friction within the gas, the equation temperature-dependent dynamic viscosity (Sutherland's Law) is used:

$$\mu = \mu_{ref} \left(\frac{T}{T_{ref}} \right)^{3/2} \frac{T_{ref} + 110}{T + 110}$$

Here, μ_{ref} is $1.846 \cdot 10^{-5}\ kg/m{\cdot}s$ at a reference temperature T_{ref} of $298\ K$. The viscous stress τ is then calculated using the velocity gradient as $\tau = (4/3)\mu(\partial u/\partial x)$, which accounts for the viscous effects along the tube in the 1D flow. Thermal conductivity k can be related to viscosity through the Prandtl number $Pr = 0.707$ using the relation $k = (C_p\mu) / Pr$, where C_p is the constant pressure specific heat. This allows for the calculation of the heat flux $q_x = -k(\partial T/\partial x)$, representing the conduction of heat across the temperature gradients produced by the shock and expansion waves.

III. Initial and Boundary Conditions

The domain of the simulation is a tube of length $L = 10$ meters, with the diaphragm being located at $x = 5$ meters. The initial conditions are set to represent regions one and four: the high-pressure driver section and a low-pressure driven section respectively, initially physically separated by the diaphragm. In the driver section, the pressure P_4 is 10^5 Pa and the density ρ_4 is 1.0 kg/m³. In the driven section, the pressure P_1 is 10^4 Pa and the density ρ_1 is 0.125 kg/m³. Initially, the gas is at rest throughout all regions of the tube (u = 0). In order to prevent numerical instability at the first time step due to the infinite gradient at the diaphragm, i.e. jump discontinuity of physical properties, the initial profiles for temperature, velocity, and pressure are smoothed over several grid points. Adiabatic boundary conditions are placed on

the ends of the tube (positions *x = 0* and *x = 10*) (i.e. zero gradient condition for temperature). A similar zero gradient condition is placed on the end walls for pressure Additionally, the fluid at the ends of the tube maintains a zero velocity per the no slip condition. These zero gradient conditions are implemented through second-order one-sided finite difference schemes derived from a Taylor series expansion (by expanding the variables at the interior points near the boundary and setting the first derivative to zero and solving for the boundary value, yielding the coefficients seen below).These conditions ensure that the gas properties at the tube ends are held constant at their initial values throughout the simulation, since wave disturbances do not reach the boundaries in the run time of the simulation.

$$\begin{cases} u = 0, & u_1 = 0, u_N = 0 \\ \frac{\partial T}{\partial x} = 0, & T_1 = \frac{4}{3}T_2 - \frac{1}{3}T_3, T_N = \frac{4}{3}T_{N-1} - \frac{1}{3}T_{N-2} \\ \frac{\partial P}{\partial x} = 0, & P_1 = \frac{4}{3}P_2 - \frac{1}{3}P_3, P_N = \frac{4}{3}P_{N-1} - \frac{1}{3}P_{N-2} \end{cases}$$

IV. Numerical Procedure and Stencil Derivations

The solver utilizes the MacCormack scheme, a predictor-corrector algorithm that provides second-order accuracy in both space and time. The method begins by calculating a predicted state at the next time level using one-sided differences, after which the corrector step is performed. The final solution is then determined by taking the average of the initial state and the predicted state, thus, the scheme achieves second-order time accuracy.

4.1 Predictor and Corrector Steps

The predictor step uses a backward difference to estimate an intermediate state vector $\overline{U}_i$:

$$\overline{U}_i = U_i^t + \Delta t \left(J_i^t - \frac{F_i^t - F_{i-1}^t}{\Delta x} \right) + S_i^t$$

After the predictor step, the velocity, pressure, density, internal energy, and temperature ($\overline{u}$, $\overline{P}$, $\overline{\rho}$, $\overline{e}$, $\overline{T}$) are extracted from $\overline{U}$ by algebraically isolating them from the mass, momentum, and energy terms. The boundary conditions are then reset so that later flux and source term calculations use the accurate physical values. The intermediate values are then used to construct the predicted source vector $\overline{J}$, flux vector $\overline{F}$, and artificial viscosity $\overline{S}$. The corrector step then determines the state at the next time level:

$$U_i^{t+\Delta t} = \frac{1}{2} \left[U_i^t + \overline{U}_i + \Delta t \left(\overline{J}_i - \frac{\overline{F}_{i+1} - \overline{F}_i}{\Delta x} \right) \right] + \overline{S}_i$$

4.2 Second-Order Stencil Derivation

To derive the MacCormack stencils, Taylor series expansions are used to solve for the coefficients in the fourth order stencil of the various functions. This allows us to construct a linear combination of function values at specific grid points to isolate the first derivative while canceling out the leading-order truncation error terms.

For the second-order forward predictor, we consider a linear combination of the flux values at position *j*, *j+1*, and *j+2*. The Taylor expansions for the forward points, expanded about x_j, are:

$$F_{j+1} = F_j + \Delta x F_j' + \frac{\Delta x^2}{2} F_j'' + \frac{\Delta x^3}{6} F_j''' + \dots$$

$$F_{j+2} = F_j + 2\Delta x F_j' + \frac{4\Delta x^2}{2} F_j'' + \frac{8\Delta x^3}{6} F_j''' + \dots$$

To achieve second-order accuracy, we require a combination of these points that eliminates the F_j" term. By multiplying the F_{j+1} expansion by *4* and subtracting the F_{j+2} expansion, the Δx^2 terms cancel. To isolate the first derivative, we subtract $3F_j$, which yields the final predictor stencil used in the code:

$$\left(\frac{\partial F}{\partial x}\right)_j = \frac{-3F_j + 4F_{j+1} - F_{j+2}}{2\Delta x} + \mathcal{O}(\Delta x^2)$$

The backward corrector is derived identically by expanding F_{j-1} and F_{j-2} in the negative direction.

4.3 Fourth-Order Gottlieb-Turkel Stencil Derivation

The fourth-order MacCormack stencil (known as the Gottlieb-Turkel or 4-2 scheme) requires a five-point stencil to cancel out error terms up to the fourth derivative. For the forward predictor, we look for coefficients for F_j, F_{j+1}, and F_{j+2}. Expanding the latter two points produces the Taylor Series expansions:

$$F_{j+1} = F_j + \Delta x F_j' + \frac{\Delta x^2}{2} F_j'' + \frac{\Delta x^3}{6} F_j''' + \frac{\Delta x^4}{24} F_j^{(4)} + \dots$$

$$F_{j+2} = F_j + 2\Delta x F_j' + \frac{4\Delta x^2}{2} F_j'' + \frac{8\Delta x^3}{6} F_j''' + \frac{16\Delta x^4}{24} F_j^{(4)} + \dots$$

By solving the system of equations to eliminate the F_j" and F_j"" errors, we obtain the specific weights: -7, 8, -1. This leads to the predictor stencil:

$$\left(\frac{\partial F}{\partial x}\right)_j = \frac{-F_{j+2} + 8F_{j+1} - 7F_j}{6\Delta x} + \mathcal{O}(\Delta x^4)$$

The corresponding corrector uses a mirrored backward approach to maintain the higher-order precision:

$$\left(\frac{\partial F}{\partial x}\right)_j = \frac{7F_j - 8F_{j-1} + F_{j-2}}{6\Delta x} + \mathcal{O}(\Delta x^4)$$

These stencils are important for high-Reynolds number flows in shock tube simulations because they help reduce the inaccurate oscillations seen near sharp gradients, unlike second-order methods. While these spatial stencils are fourth-order, the overall time integration remains second-order due to the MacCormack averaging process.

V. Artificial Viscosity and Stability

An important component of this simulation is the use of artificial viscosity, *S*. High-order schemes are prone to non-physical oscillations (known as the Gibbs phenomenon) near sharp discontinuities like shock waves. To suppress the oscillations, a pressure-weighted damping term is added to the predictor and corrector steps. The term S_i^t is defined as:

$$S_i^t = C_x \frac{|P_{i+1}^t - 2P_i^t + P_{i-1}^t|}{P_{i+1}^t + 2P_i^t + P_{i-1}^t}(U_{i+1}^t - 2U_i^t + U_{i-1}^t)$$

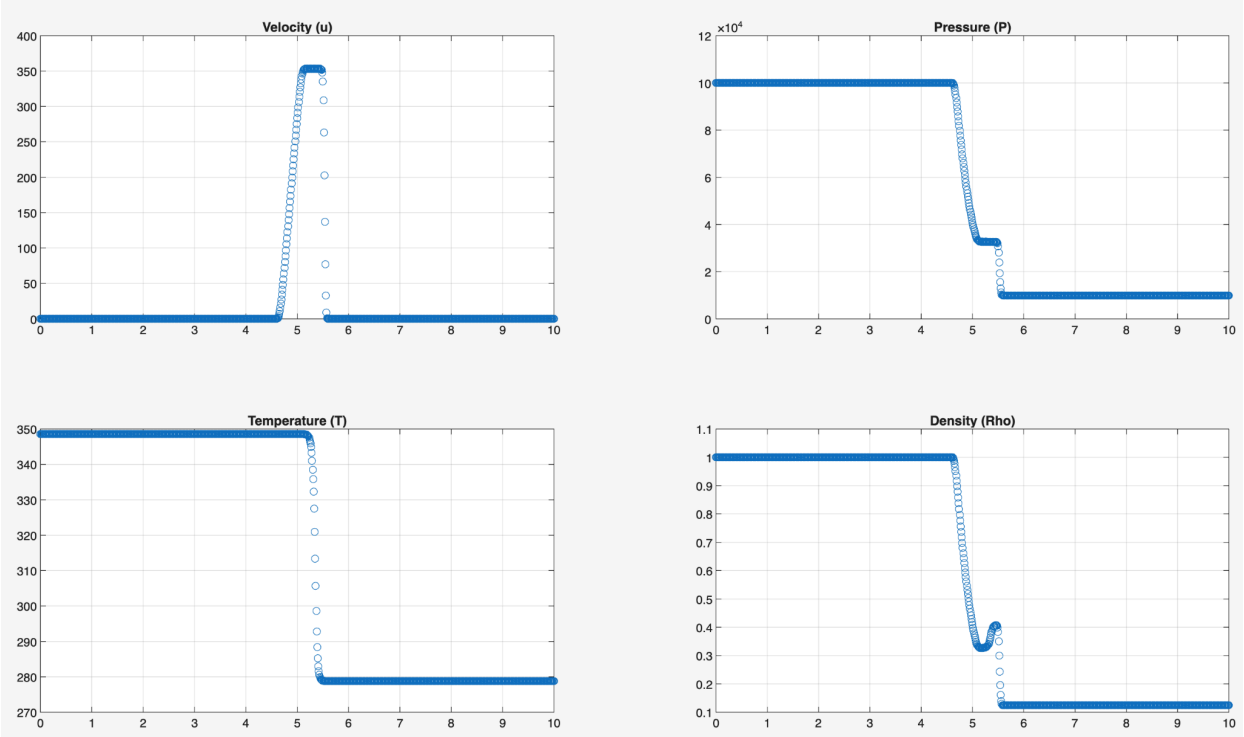
The coefficient C_x controls the strength of the damping. Per the assignment guidelines, values of C_X were tested within the range of 0.01 to 0.3, starting with $C_x = 0.1$. This term works by increasing local diffusion where the pressure gradient is high while preserving accuracy in smooth regions.

VI. Results and Discussion

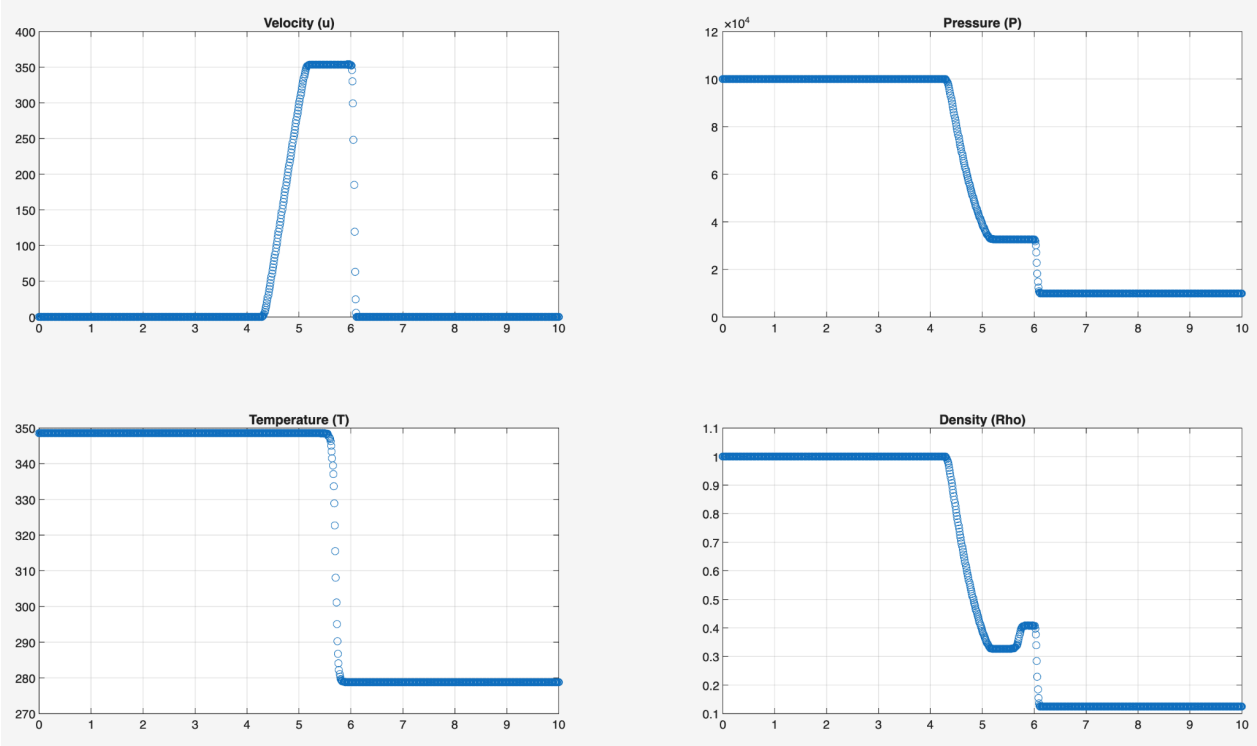
6.1 Time Evolution of Flow Properties

To demonstrate the time-dependent behavior of the shock tube system, the viscous fourth-order solver was used to track the profiles of flow variables in the tube at multiple timestamps (0.001 to 0.006 seconds). The resulting profiles for pressure, density, velocity, and temperature demonstrate the spreading out of the wave structures from the initial diaphragm location, with the shock front and contact surface propagating into the low-pressure region while the expansion wave widens into the high-pressure driver section. This can be seen by the motion of characteristic regions in the graphs where the fluid properties change abruptly.

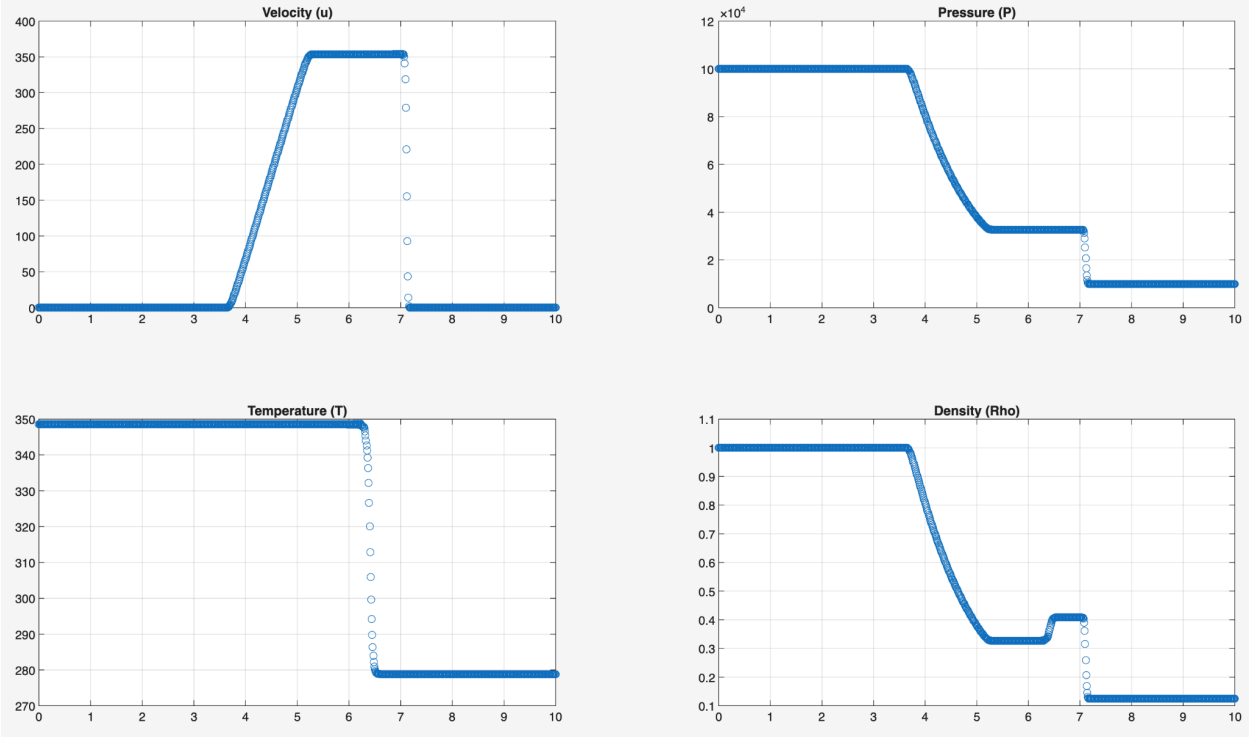
(A)



(B)


Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)

(C)


Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)

(D)

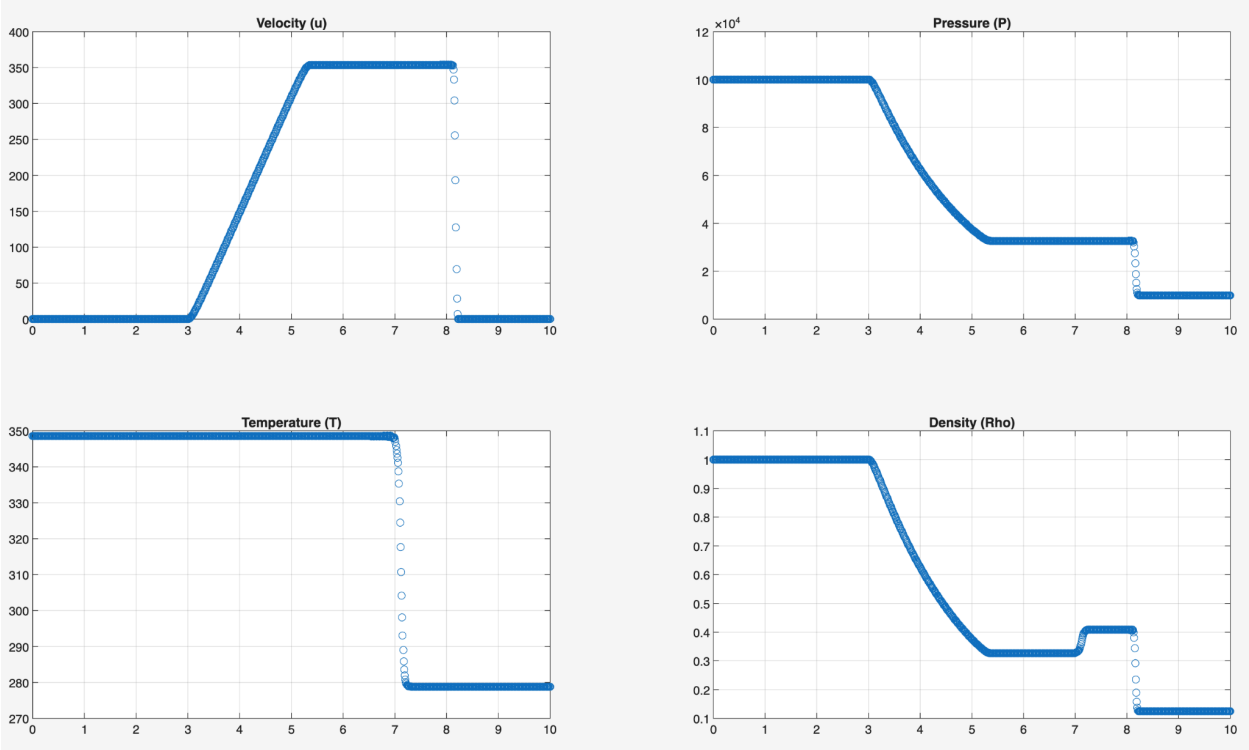


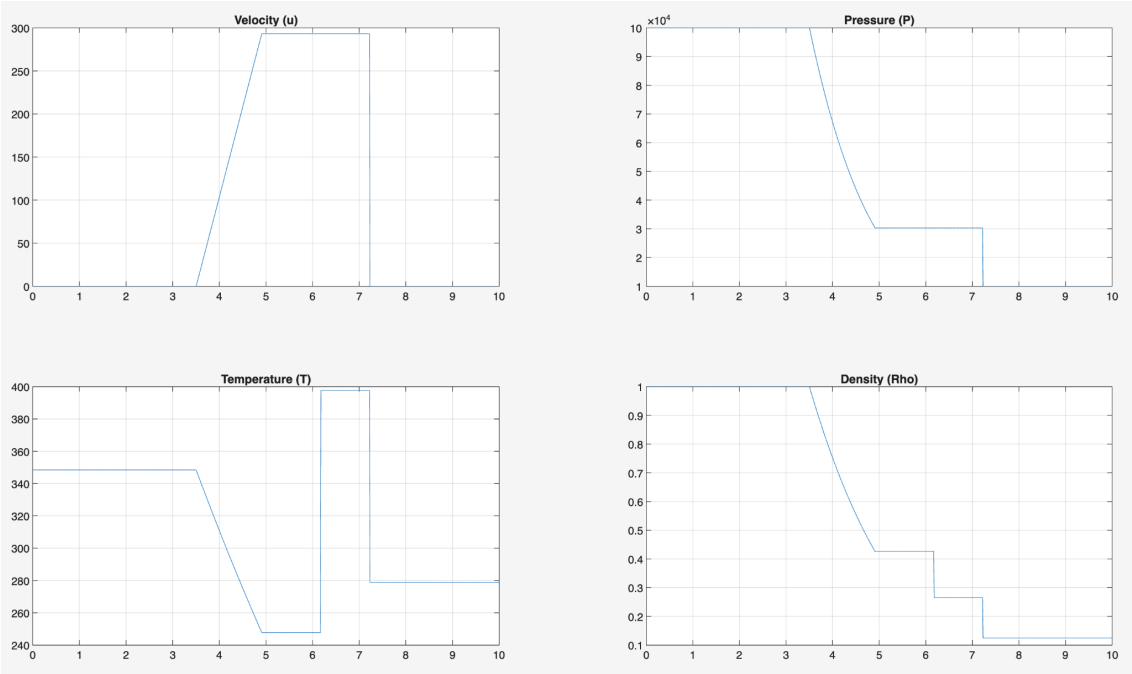
Fig 2. Spatial distribution of pressure, density, velocity, and temperature at (A) t = 0.001, (B) t = 0.002, (C) t = 0.004, and (D) t = 0.006 seconds. The profiles, generated using the viscous fourth-order solver (N_X=1001)

6.2 Comparison of Inviscid Analytical and Viscous Numerical Solutions

The first major observation in this simulation is the contrast between the ideal analytical solution and the real viscous numerical results. The differences between the two highlight how physical dissipation and thermal diffusion smoothen the representation of fluid discontinuities. In the density profile, while the analytical solution displays a series of sharp step-decreases at the expansion wave and contact surface, the viscous numerical results show a distinct density increase at the second step. This occurs because the numerical model accounts for thermal conduction, which requires a local rise in density to maintain pressure equilibrium as the temperature profile is smoothened. Additionally, in the temperature profiles, the analytical solution displays a downward ramp followed by a sharp upward step at the contact surface while the viscous case exhibits a broader downward step across the tube length. This change is driven by the conduction and viscosity terms in the source vector *J*, which allow energy to diffuse away from the interface rather than remaining trapped in the sharp jump discontinuity, yielding a more accurate depiction of how real gases behave when hot and cold layers interact. Moreover, the regions of sharp increase or decrease in the velocity, pressure, and density profiles appear less instantaneous in the viscous case and exhibit a visible slope compared to the analytical inviscid solution. This smoothing is due to the inclusion of viscosity and thermal conductivity, which are diffusive mechanisms that smooth discontinuities across several grid points to simulate the internal friction and heat transfer within the gas. Note that the fourth-order system is used for this comparison because its better spatial resolution minimizes numerical

dissipation, making sure that the observed smoothing of discontinuities is a true reflection of physical viscosity and thermal conduction rather than an artifact of numerical error.

(A)



(B)

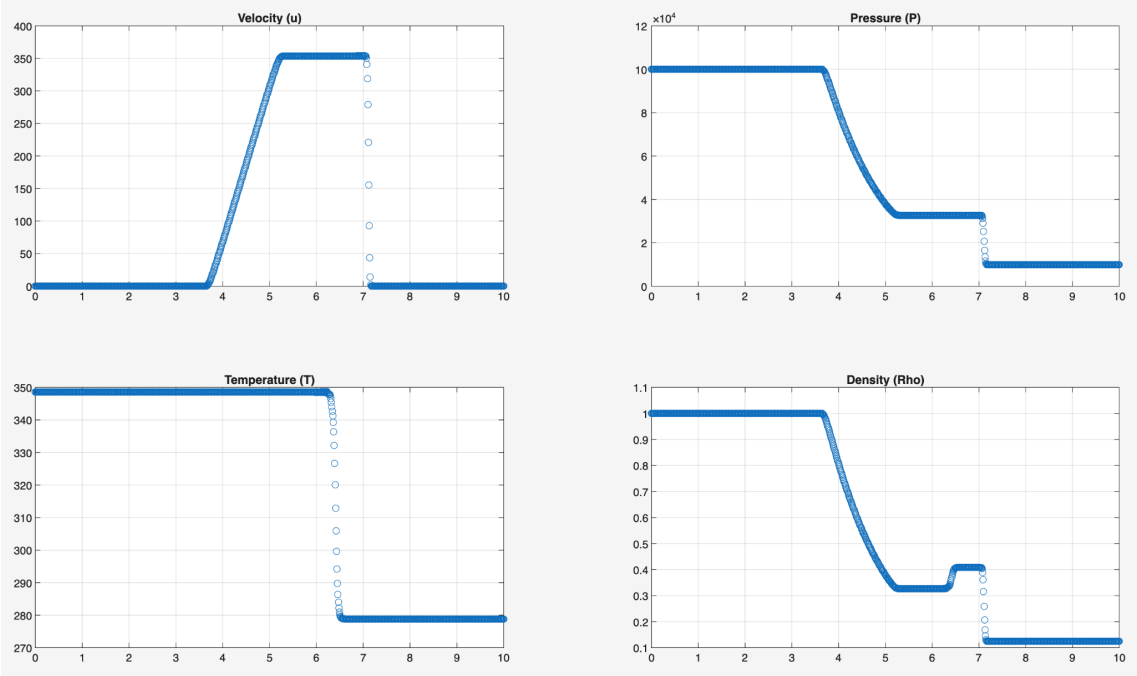
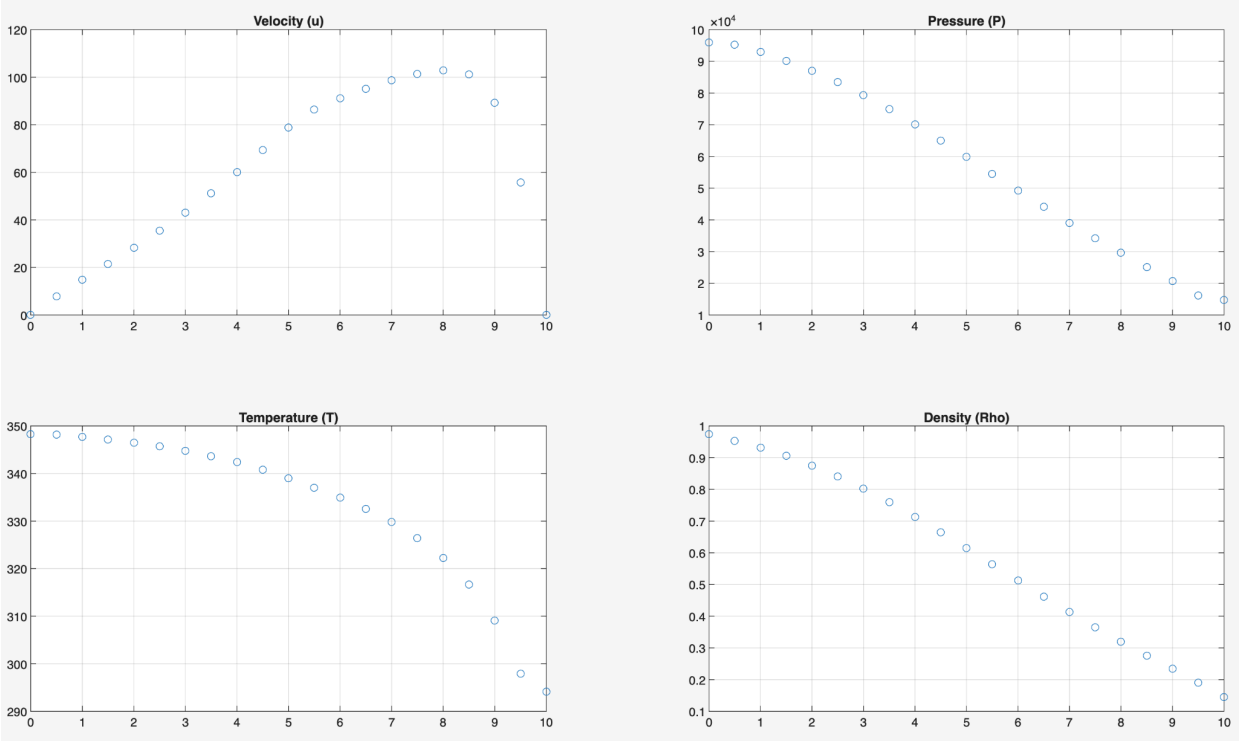


Fig 3. Spatial profiles of pressure, density, velocity, and temperature at t=0.004 s, comparing the (A) inviscid analytical solution to the (B) viscous fourth-order numerical results. N_X=1001, C_X=0.1.

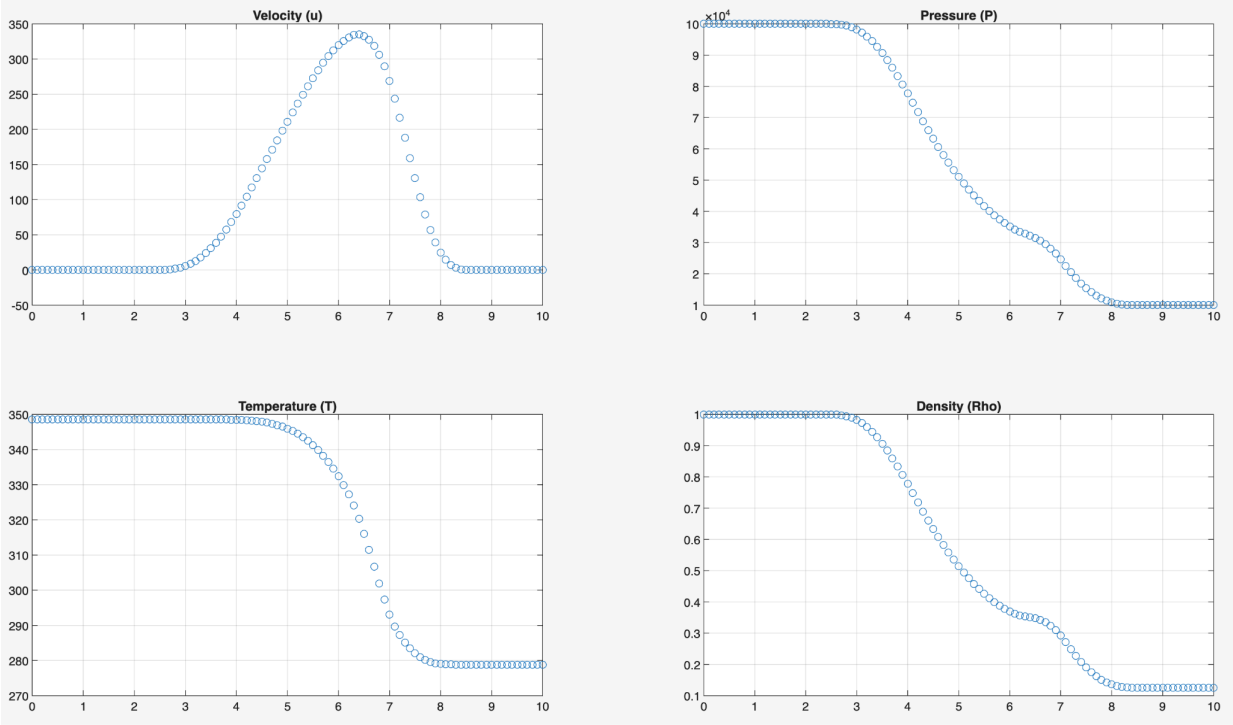
6.3 Grid Convergence Study

To determine the sensitivity of the model to spatial discretization, the simulation was executed at three different grid densities: $N_X = 21, 101, 1001$. As the number of grid points increases, it is observed that the numerical solution converges toward the analytical inviscid solution (see Fig. 3). When $N_X = 21$, the shock front and contact discontinuity are significantly smeared and instantaneous changes are smoothened. On the other hand, at $N_X = 1001$, the jumps are much sharper and the wave speeds are more accurately resolved. This demonstrates that while viscosity provides physical smearing, numerical error provides additional non-physical smearing that can be mitigated only by a finer mesh/grid.

(A)



(B)



(C)

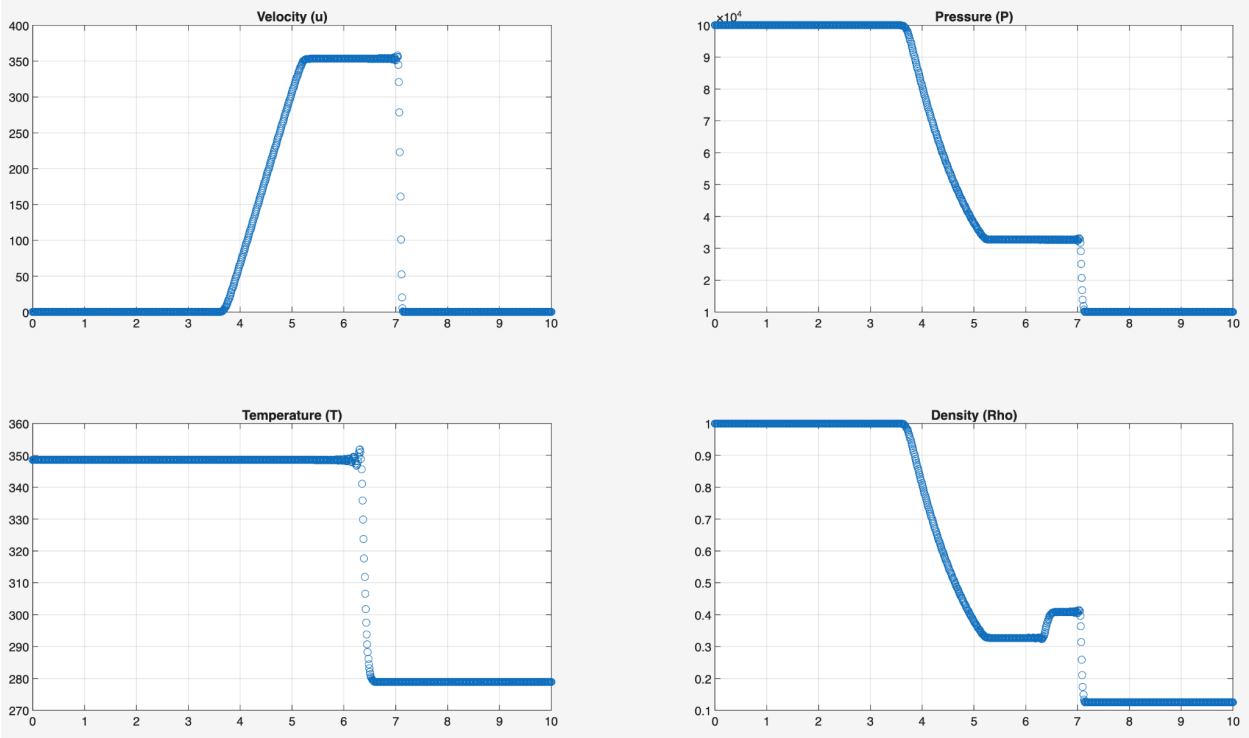
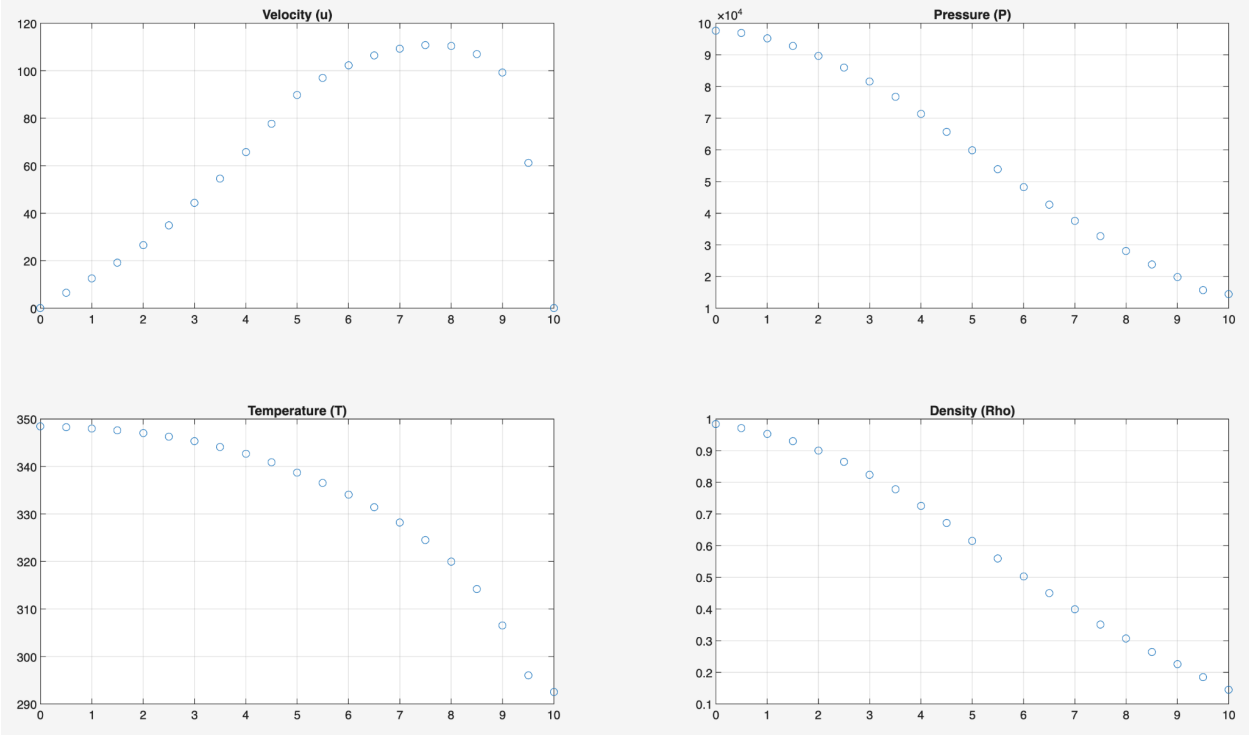


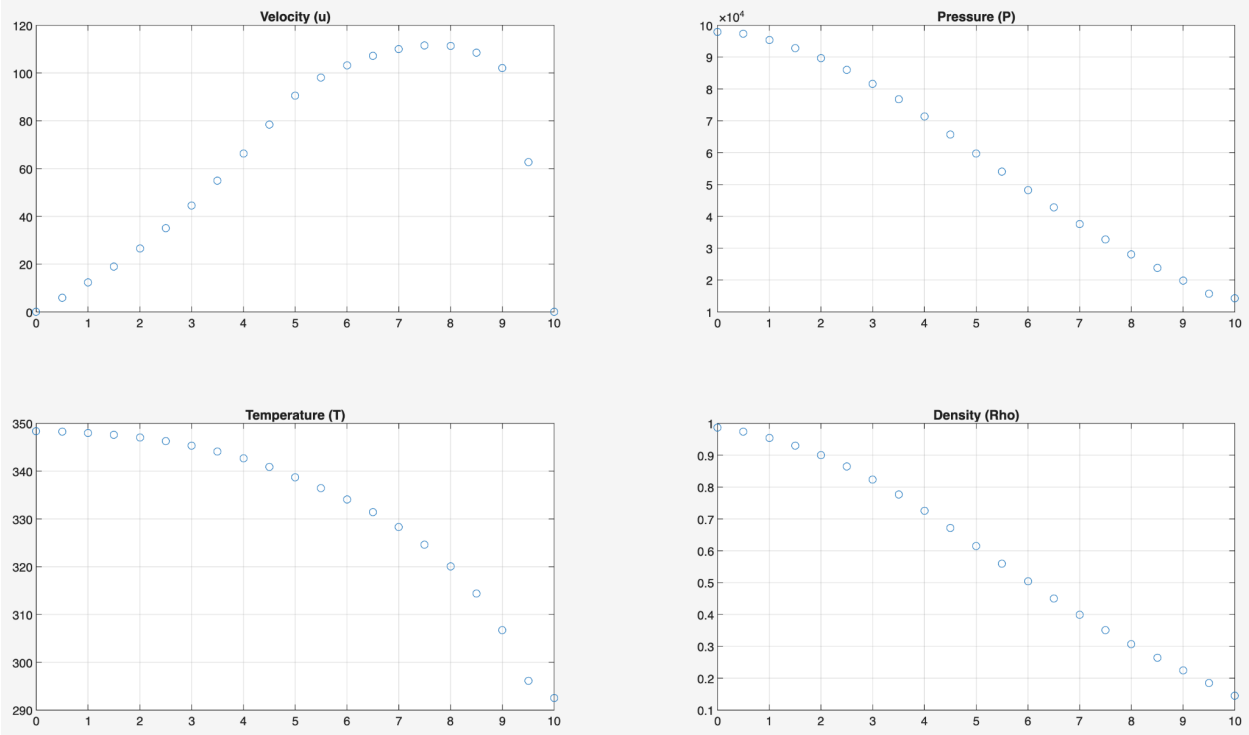
Fig 4. Influence of grid resolution on the numerical solution of 2nd order viscous solver at t = 0.004 s, C_X = 0.1. A comparison of (A) N_X = 21, (B) N_X = 101, and (C) N_X = 1001.

6.4 Comparison of Second and Fourth-Order Schemes

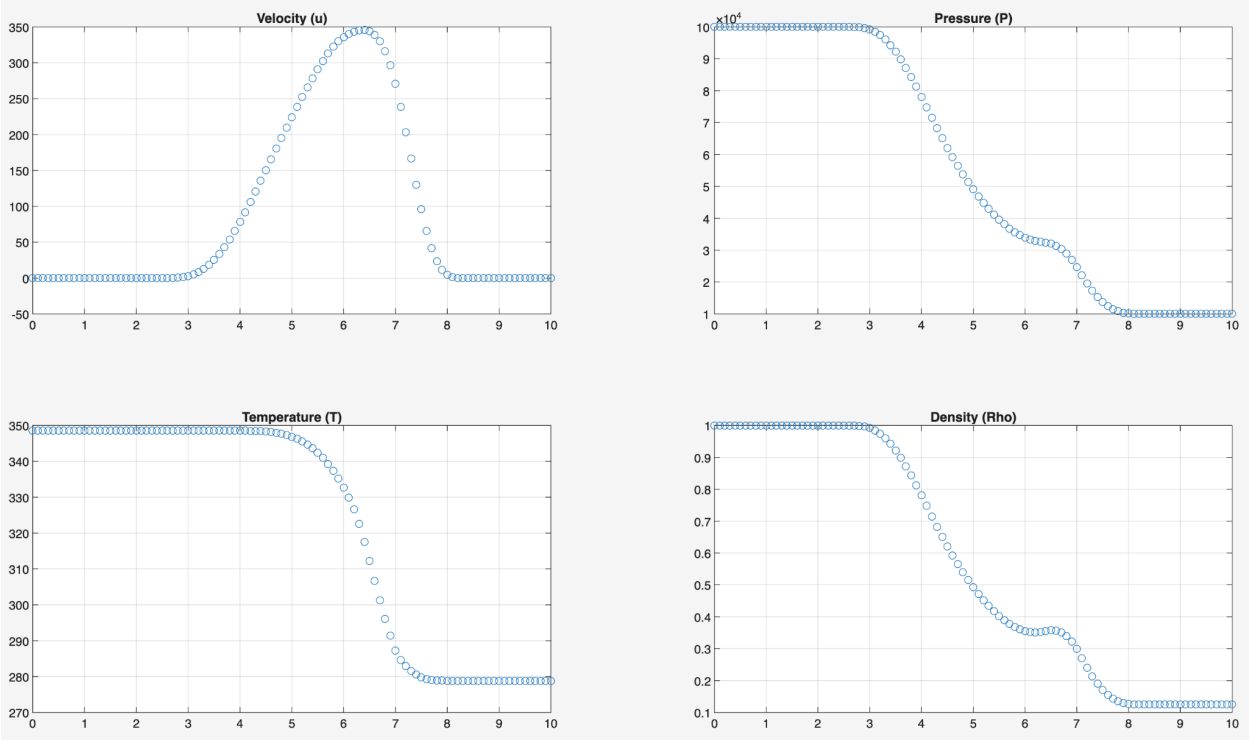
Interestingly, a comparison between the second-order MacCormack scheme and the fourth-order Gottlieb-Turkel scheme shows that the results remain virtually identical across coarse and fine meshes. This similarity occurs because the shock tube problem is characterized by sharp discontinuities, where high-order schemes must locally resort to using lower-order accuracy to prevent unphysical oscillations and maintain stability. Furthermore, the inclusion of physical viscosity and the application of artificial viscosity, C_X, provide a dissipative effect that smooths the profiles across the grid. Since this smoothing is intentionally designed to span a specific number of grid points in order to ensure stability, the added spatial precision of the fourth-order stencil is overwhelmed by the dissipation implemented which is required to resolve the shock, leading to a solution governed more by the physics of the viscous terms than by the order of the numerical solver scheme. At a high resolution of N_X=1001, a slight distinction can be seen as the second-order solver shows a mild overshoot at the top of each step-down in the velocity, pressure, temperature, and density profiles. In contrast, the fourth-order scheme depicts cleaner, more defined corners at these transitions, demonstrating its improved ability to prevent errors even when the overall tube profiles are extremely similar.

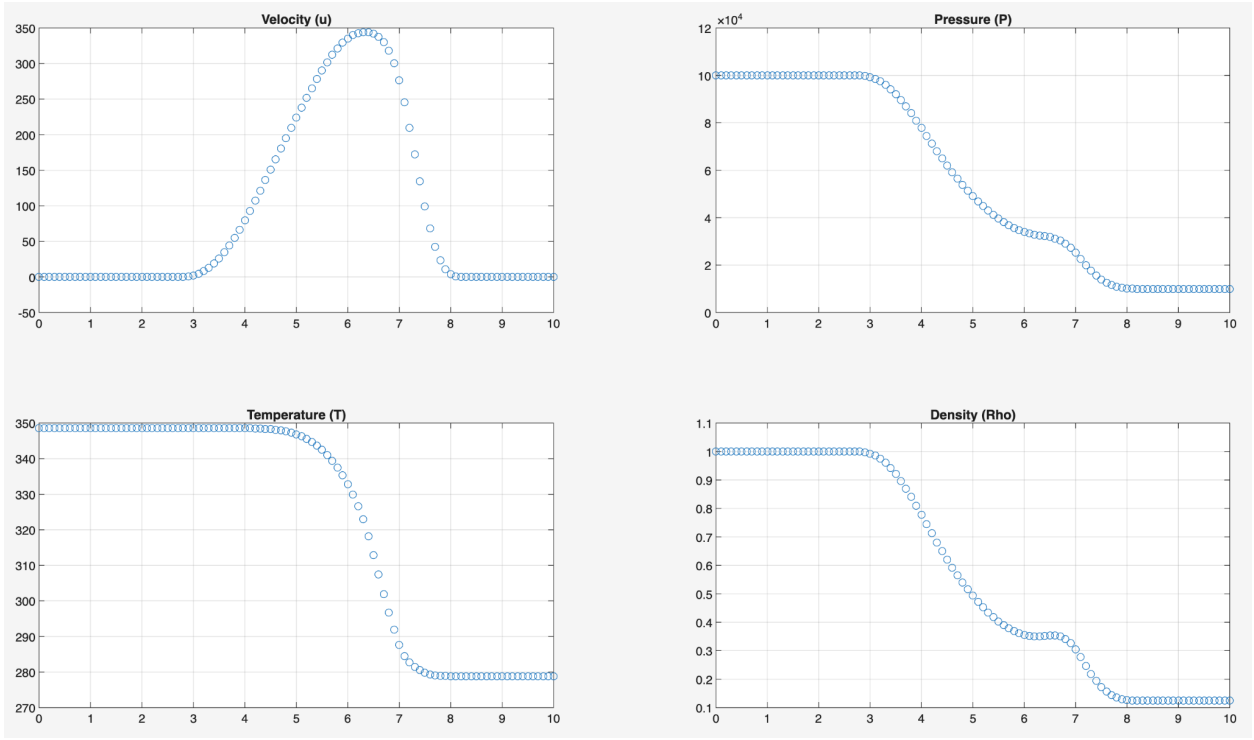
(A)



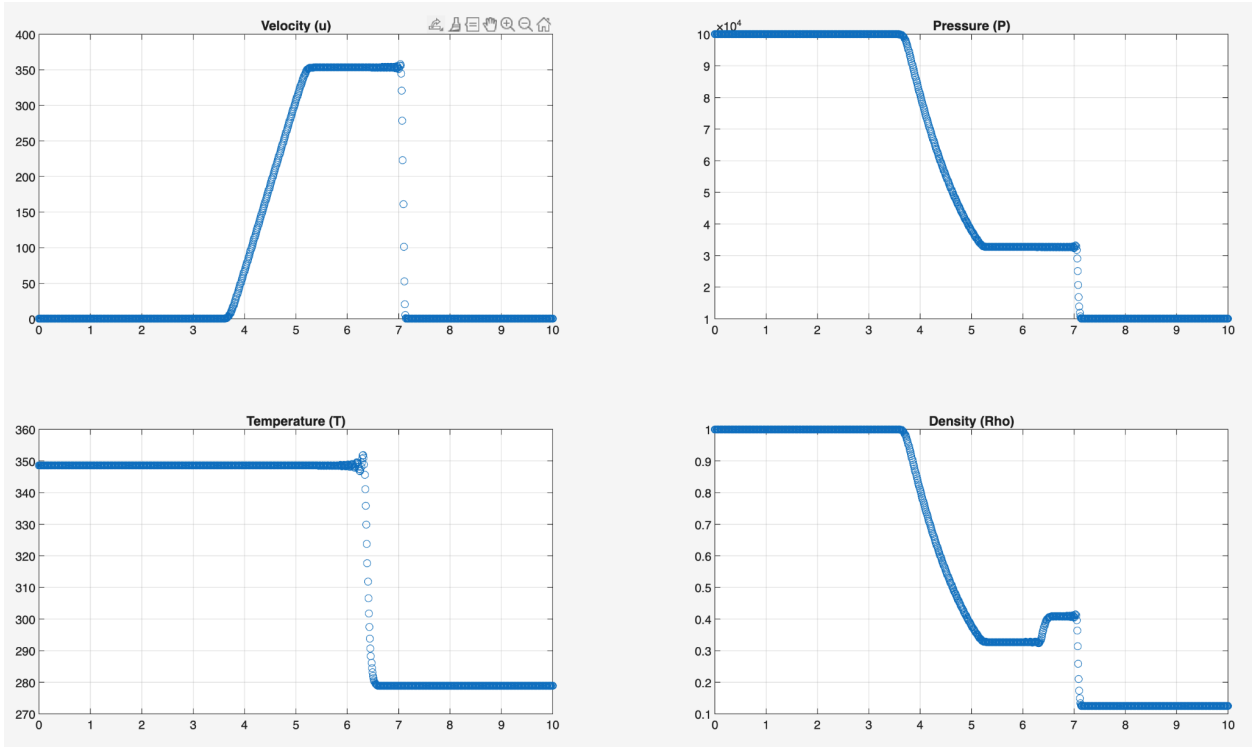

Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)

(B)


Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)


Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)

(C)


Velocity (u)
Pressure (P)
×10^4
Temperature (T)
Density (Rho)

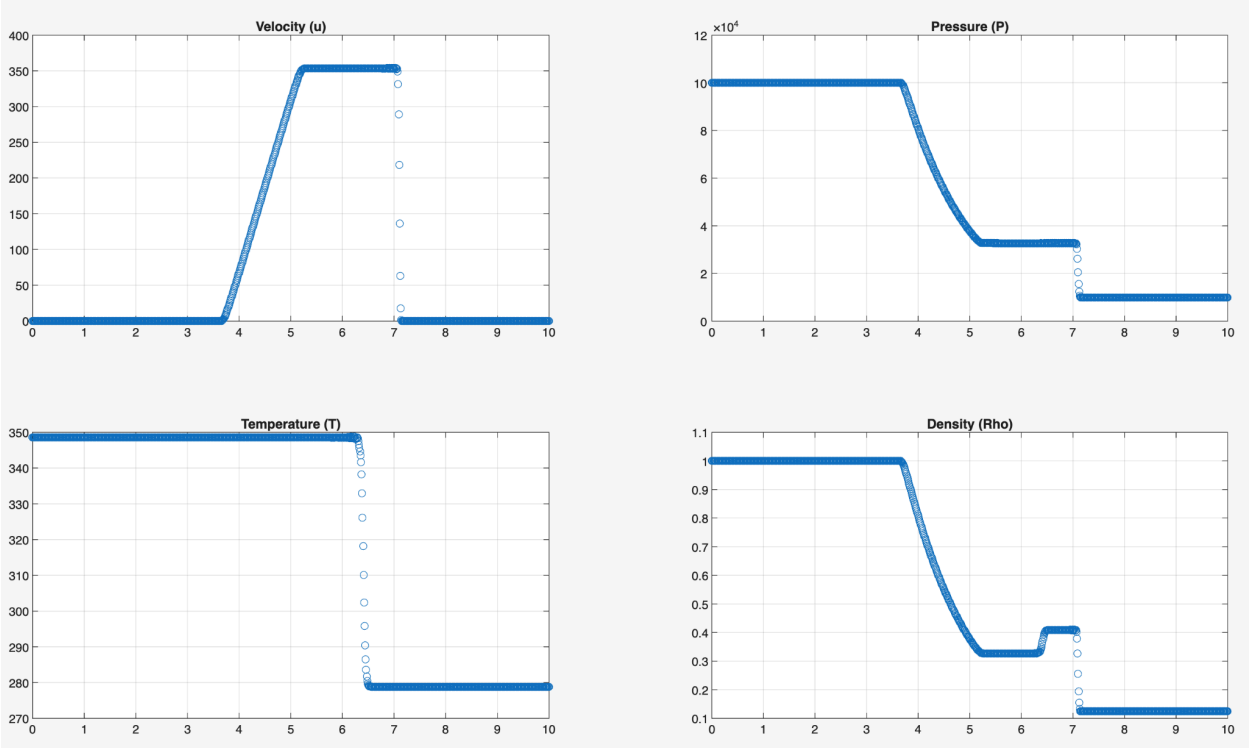


Fig 5. Comparison of 2nd order and 4th order accuracy schemes across varying grid densities, (A) N_X=21, C_x=0.05, (B) N_X=101, C_x=0.05, (C) N_X=1001, C_x=0.05 and t = 0.004 seconds using viscous solvers.

VII. Conclusions

This simulation was able to implement a MacCormack-based solver for the unsteady 1D Navier-Stokes equations. By incorporating temperature-dependent dynamic viscosity and thermal conduction, the model provided a realistic view of wave propagation in a shock tube. The study confirmed that while fourth-order schemes provide superior theoretical accuracy, their practical advantage is limited in flows dominated by strong shocks and artificial damping (artificial viscosity, C_X). The convergence of the numerical model toward the 'true' analytical solution with increased grid resolution validates the code.

VIII. Appendix: MATLAB Code

MATLAB Code for Fourth-Order Viscous Case:

```
% ME193 Project I - 1D Unsteady Compressible Navier-Stokes
% Implement the Fourth Order MacCormack scheme to solve the Shock Tube problem
%% Constants and Parameters
Lx=10; % Domain length in meters
xo = 5; % Diaphragm location
Cp=1.005E3; % Specific heat at constant pressure
Cv=0.718E3; % Specific heat at constant volume
Cx = 0.05; % Artificial viscosity coefficient (Try Cx approx 0.1 first)
Gamma=Cp/Cv; % Ratio of heats (1.4 for air)
Pr=0.707; % Prandtl number
R=286.9; % Gas constant
Mu_ref=1.846E-5; % Reference viscosity
```

```
T_ref=298; % Reference temperature
%% Initial Conditions
% For State 1 (Driven)
P1 = 1E4;
Rho1 = 0.125;
% For State 4 (Driver)
P4 = 1E5;
Rho4 = 1;
%% Grid and Time Generation
time_step = 1E-7; % Time step size (better for convergence)
time_end = 4E-3; % Total simulation time
time = 0:time_step:time_end;
Nx = 1001; % Number of grid points
x = 0:(Lx/(Nx-1)):Lx; % Grid generation
dx = x(2)-x(1);
%% Initialization of Vectors
% P=Pressure, Rho=Density, u=Velocity, T=Temperature, e=Internal Energy
[P, Rho, u, T, Tao, qx, K, Mu, Lambda,e] = deal(zeros(length(x), length(time)));
% U=Conserved State Vector, F=Flux Vector, J=Source Vector
[U,F,J] = deal(zeros(length(x), length(time), 3));
% Predicted values (denoted with _)
[Rho_,u_,e_,P_,T_, Mu_, Lambda_, Tao_, K_, qx_] = deal(zeros(length(x),1));
[U_,J_,F_, S, S_] = deal(zeros(length(x),3));
% Setting initial conditions across the diaphragm
initialpoint = round((xo/Lx)*(Nx-1))+1;
P(1:initialpoint, 1) = P4;
P(initialpoint:end, 1) = P1;
Rho(1:initialpoint, 1) = Rho4;
Rho(initialpoint:end, 1) = Rho1;
% Initial internal energy and temperature calculation
T(:,1) = P(:,1) ./ (R*Rho(:,1)); % Ideal Gas Law
e(:,1) = Cv * T(:,1); % Internal Energy
% Smoothing the initial jump over 2-3 grid points to improve stability
T(:,1) = smooth(T(:,1));
u(:,1) = smooth(u(:,1));
P(:,1) = smooth(P(:,1));
Rho(:,1) = smooth(Rho(:,1));
e(:,1) = smooth(e(:,1));
%% Initial Plots
% Plots of the velocty, pressure, temperature, and density profiles along
% the tube before the shock wave
figure('Name', 'Shock Tube Initial Conditions');
subplot(221)
plot(x,u(:,1),'o');title('Initial Velocity (u)')
subplot(222)
plot(x,P(:,1),'o');title('Initial Pressure (P)')
subplot(223)
plot(x,T(:,1),'o');title('Initial Temperature (T)')
subplot(224)
plot(x,Rho(:,1),'o');title('Initial Density (Rho)')
%% Main Time Loop (MacCormack Scheme)
% i = time counter
% j = distance counter
for i = 1:length(time)
```

```
   % Defining viscous terms (using given equations)
   Mu(:,i) = Mu_ref * (T(:,i)/T_ref).^(1.5) .* (T_ref + 110) ./ (T(:,i) + 110); %
Sutherland's Law for Viscosity
   Lambda(:,i) = -(2/3) * Mu(:,i); % Stokes' Hypothesis
   K(:,i) = (Cp * Mu(:,i)) / Pr; % Thermal conductivity
   % Distance loop for Source Vector J
   for j = 1:length(x)
       if j == 1
           du = (-3*u(j,i)+4*u(j+1,i)-u(j+2,i)) / (2*(dx));
           dT = (-3*T(j,i)+4*T(j+1,i)-T(j+2,i)) / (2*(dx));
       elseif j == length(x)
           du = (3*u(j,i)-4*u(j-1,i)+u(j-2,i)) / (2*(dx));
           dT = (3*T(j,i)-4*T(j-1,i)+T(j-2,i)) / (2*(dx));
       else
           du = (u(j+1,i) - u(j-1,i)) / (2*dx); % Central difference
           dT = (T(j+1,i) - T(j-1,i)) / (2*dx);
       end
       Tao(j,i) = (4/3) * Mu(j,i) * du; % Viscous Stress
       qx(j,i)  = -K(j,i) * dT; % Heat Flux
   end
   % Source Vector J Calculation
   % Calculate the gradient of the fluxes we just found
   for j = 1:length(x)
       if j > 1 && j < length(x)
           % Momentum Source: J2 = d(Tao)/dx
           J(j,i,2) = (Tao(j+1,i) - Tao(j-1,i)) / (2*dx);

           % Energy Source: J3 = d(u*Tao - qx)/dx
           J(j,i,3) = ((u(j+1,i)*Tao(j+1,i) - qx(j+1,i)) - (u(j-1,i)*Tao(j-1,i) -
qx(j-1,i))) / (2*dx);
       else
           % Boundary sources (can be set to 0 or use one-sided differences)
           J(j,i,:) = 0;
       end
   end
   % Define Conserved State Vector U and Flux Vector F
   U(:,i,1) = Rho(:,i); % Row 1: rho
   U(:,i,2) = Rho(:,i) .* u(:,i); % Row 2: rho*u
   U(:,i,3) = Rho(:,i) .* e(:,i); % Row 3: rho*e
   F(:,i,1) = Rho(:,i) .* u(:,i); % Row 1: rho*u
   F(:,i,2) = Rho(:,i) .* u(:,i).^2 + P(:,i); % Row 2: rho*u^2 + P
   F(:,i,3) = (Rho(:,i) .* e(:,i)) .* u(:,i); % Row 3: (rho*e)*u

   % Predictor Step
   if i ~= length(time)
       for j = 1:length(x)
           % Artificial Viscosity S for the Predictor Step
           if j > 1 && j < length(x)
               P_grad = abs(P(j+1,i)-2*P(j,i)+P(j-1,i)) /
(P(j+1,i)+2*P(j,i)+P(j-1,i));
               S(j,1) = Cx * P_grad * (U(j+1,i,1) - 2*U(j,i,1) + U(j-1,i,1));
               S(j,2) = Cx * P_grad * (U(j+1,i,2) - 2*U(j,i,2) + U(j-1,i,2));
               S(j,3) = Cx * P_grad * (U(j+1,i,3) - 2*U(j,i,3) + U(j-1,i,3));
           else
               S(j,:) = 0;
```

```
    end

    % Fourth Order Spatial Difference (Forward)
    if j == 1
        % Left Boundary: 2nd order one-sided forward difference
        dF1_dx = (-3*F(j,i,1) + 4*F(j+1,i,1) - F(j+2,i,1)) / (2*dx);
        dF2_dx = (-3*F(j,i,2) + 4*F(j+1,i,2) - F(j+2,i,2)) / (2*dx);
        dF3_dx = (-3*F(j,i,3) + 4*F(j+1,i,3) - F(j+2,i,3)) / (2*dx);
    elseif j == 2
        % Fallback to 2nd order forward
        dF1_dx = (F(j+1,i,1) - F(j,i,1)) / dx;
        dF2_dx = (F(j+1,i,2) - F(j,i,2)) / dx;
        dF3_dx = (F(j+1,i,3) - F(j,i,3)) / dx;
    elseif j >= length(x)-1
        % Maintain boundary state gradient
        dF1_dx = 0;
        dF2_dx = 0;
        dF3_dx = 0;
    else
        % 4th Order Gottlieb-Turkel Stencil
        dF1_dx = (-F(j+2,i,1) + 8*F(j+1,i,1) - 7*F(j,i,1)) / (6*dx);
        dF2_dx = (-F(j+2,i,2) + 8*F(j+1,i,2) - 7*F(j,i,2)) / (6*dx);
        dF3_dx = (-F(j+2,i,3) + 8*F(j+1,i,3) - 7*F(j,i,3)) / (6*dx);
    end
    if j == length(x)
        % Right Boundary: Maintain current state
        U_(j,1) = U(j,i,1);
        U_(j,2) = U(j,i,2);
        U_(j,3) = U(j,i,3);
    else
        U_(j,1) = U(j,i,1) + time_step * (J(j,i,1) - dF1_dx) + S(j,1);
        U_(j,2) = U(j,i,2) + time_step * (J(j,i,2) - dF2_dx) + S(j,2);
        U_(j,3) = U(j,i,3) + time_step * (J(j,i,3) - dF3_dx) + S(j,3);
    end
end
% Predictor Decode: Extract primitive variables from U_
Rho_(:) = U_(:,1);
u_(:) = U_(:,2) ./ Rho_(:);
e_(:) = U_(:,3) ./ Rho_(:);
T_(:) = e_(:) / Cv; % Internal energy relation
P_(:) = Rho_(:) .* R .* T_(:); % Ideal gas law
% Viscous corrector prep (predicted step)
Mu_(:) = Mu_ref * (T_(:)/T_ref).^(1.5) .* (T_ref+110) ./ (T_(:)+110);
K_(:) = (Cp * Mu_(:)) / Pr;

% Update Predicted Source J_ and Flux F_
for j = 1:length(x)
    if j == 1
        du_p = (-3*u_(j)+4*u_(j+1)-u_(j+2)) / (2*dx);
        dT_p = (-3*T_(j)+4*T_(j+1)-T_(j+2)) / (2*dx);
    elseif j == length(x)
        du_p = (3*u_(j)-4*u_(j-1)+u_(j-2)) / (2*dx);
        dT_p = (3*T_(j)-4*T_(j-1)+T_(j-2)) / (2*dx);
    else
        du_p = (u_(j+1) - u_(j-1)) / (2*dx);
```

```
            dT_p = (T_(j+1) - T_(j-1)) / (2*dx);
        end
        Tao_(j,1) = (4/3) * Mu_(j) * du_p;
        qx_(j,1)  = -K_(j) * dT_p;
    end
    for j = 1:length(x)
        if j > 1 && j < length(x)
            J_(j,2) = (Tao_(j+1,1) - Tao_(j-1,1)) / (2*dx);
            J_(j,3) = ((u_(j+1)*Tao_(j+1,1) - qx_(j+1,1)) - (u_(j-1)*Tao_(j-1,1) -
qx_(j-1,1))) / (2*dx);
        else
            J_(j,:) = 0;
        end
    end

    F_(:,1) = Rho_(:) .* u_(:);
    F_(:,2) = Rho_(:) .* u_(:).^2 + P_(:);
    F_(:,3) = (Rho_(:) .* e_(:)) .* u_(:);

    % Corrector Step
    for j = 1:length(x)
        % Artificial Viscosity S_bar for the Corrector Step
        if j > 1 && j < length(x)
            P_grad_p = abs(P_(j+1)-2*P_(j)+P_(j-1)) / (P_(j+1)+2*P_(j)+P_(j-1));
            S_(j,1) = Cx * P_grad_p * (U_(j+1,1) - 2*U_(j,1) + U_(j-1,1));
            S_(j,2) = Cx * P_grad_p * (U_(j+1,2) - 2*U_(j,2) + U_(j-1,2));
            S_(j,3) = Cx * P_grad_p * (U_(j+1,3) - 2*U_(j,3) + U_(j-1,3));
        else
            S_(j,:) = 0;
        end
        % Fourth Order Spatial Difference (Backward)
        if j == 1
            % Maintain boundary
            U(j,i+1,1) = U_(j,1);
            U(j,i+1,2) = U_(j,2);
            U(j,i+1,3) = U_(j,3);
            continue; % Move to next j
        elseif j == length(x)
            % Right Boundary: 2nd order one-sided backward difference
            dF1_dx_p = (3*F_(j,1) - 4*F_(j-1,1) + F_(j-2,1)) / (2*dx);
            dF2_dx_p = (3*F_(j,2) - 4*F_(j-1,2) + F_(j-2,2)) / (2*dx);
            dF3_dx_p = (3*F_(j,3) - 4*F_(j-1,3) + F_(j-2,3)) / (2*dx);
        elseif j == length(x)-1
            % Fallback to 2nd order backward
            dF1_dx_p = (F_(j,1) - F_(j-1,1)) / dx;
            dF2_dx_p = (F_(j,2) - F_(j-1,2)) / dx;
            dF3_dx_p = (F_(j,3) - F_(j-1,3)) / dx;
        elseif j <= 2
            % Handled by left boundary maintenance
            dF1_dx_p = 0;
            dF2_dx_p = 0;
            dF3_dx_p = 0;
        else
            % 4th Order Backward Stencil
            dF1_dx_p = (7*F_(j,1) - 8*F_(j-1,1) + F_(j-2,1)) / (6*dx);
```

```
            dF2_dx_p = (7*F_(j,2) - 8*F_(j-1,2) + F_(j-2,2)) / (6*dx);
            dF3_dx_p = (7*F_(j,3) - 8*F_(j-1,3) + F_(j-2,3)) / (6*dx);
        end
        if j > 1
            U(j,i+1,1) = 0.5 * (U(j,i,1) + U_(j,1) + time_step * (J_(j,1) -
dF1_dx_p)) + S_(j,1);
            U(j,i+1,2) = 0.5 * (U(j,i,2) + U_(j,2) + time_step * (J_(j,2) -
dF2_dx_p)) + S_(j,2);
            U(j,i+1,3) = 0.5 * (U(j,i,3) + U_(j,3) + time_step * (J_(j,3) -
dF3_dx_p)) + S_(j,3);
        end
    end

    % Final Decode for next time step
    Rho(:,i+1) = U(:,i+1,1);
    u(:,i+1) = U(:,i+1,2) ./ Rho(:,i+1);
    e(:,i+1) = U(:,i+1,3) ./ Rho(:,i+1);
    T(:,i+1) = e(:,i+1) / Cv;
    P(:,i+1) = Rho(:,i+1) .* R .* T(:,i+1);
    % Apply Boundary Conditions for Pressure and Temperature
    u(1,i+1) = 0;
    u(end,i+1) = 0;
    P(1,i+1) = (4/3)*P(2,i+1) - (1/3)*P(3,i+1);
    P(end,i+1) = (4/3)*P(end-1,i+1) - (1/3)*P(end-2,i+1);
    T(1,i+1) = (4/3)*T(2,i+1) - (1/3)*T(3,i+1);
    T(end,i+1) = (4/3)*T(end-1,i+1) - (1/3)*T(end-2,i+1);
  end
end
%% Final Plots
% Plots of the velocty, pressure, temperature, and density profiles along
% the tube 0.004 seconds after the diaphram breaks
figure('Name', 'Viscous Fourth Order Results');
subplot(221)
plot(x,u(:,i),'o');title('Velocity (u)')
grid on;
subplot(222)
plot(x,P(:,i),'o');title('Pressure (P)')
grid on;
subplot(223)
plot(x,T(:,i),'o');title('Temperature (T)')
grid on;
subplot(224)
plot(x,Rho(:,i),'o');title('Density (Rho)')
grid on;
function T2 = smooth(T)
T2 = zeros(size(T));
T3 = zeros(size(T));
T2(2:end) = 0.5*(T(2:end)+T(1:end-1));T2(1) = T(1);
T3(3:end) = 0.5*(T2(3:end)+T2(2:end-1));T3(1:2) = T(1:2);
end
```